OFFSIDE LABS

Reflect Proxy

Smart Contract Security Assessment

November 2025

Prepared for:

Reflect

Prepared by:

Offside Labs

Sirius Xie

Ripples Wen



Contents

1 About Offside Labs **2**

2 Executive Summary **3**

3 Summary of Findings **5**

4 Key Findings and Recommendations **6**

4.1 Incorrect branded_mint Amount Lead to User loss 6
4.2 Crank Operation Failure Due to Over Claim Commission Fee 7
4.3 Missing Slippage Checks in Several IXs 8
4.4 DoS Risk When the Oracle Price Falls 9
4.5 Possible Inflation Attack on Wrap IX 10
4.6 Inconsistent Interest Split Calculation between Code Comment and Implementation 11
4.7 DoS Risk in CreateProxy Account Creation 13
4.8 Missing Overflow Check in Arithmetical Operation 14
4.9 Missing Validation on mint_to amount May Lead to User Loss 14
4.10 Potential Overflow in Intermediate Multiplication Despite Valid Final Result . . 15
4.11 Informational and Undetermined Issues 16

5 Disclaimer **17**

1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.


https://offside.io/


https://github.com/offsidelabs


https://twitter.com/offside_labs

2 Executive Summary

Introduction

Offside Labs completed a security audit of *Reflect Proxy* smart contracts, starting on September 22th, 2025, and concluding on November 7th, 2025.

Project Overview

Reflect Proxy enables integrators to wrap user-held USDC+ into branded stablecoins and unwrap them back while maintaining principal and integrator-fee balances in a shared state account. A permissioned oracle feeds USDC+/USDC prices for wrap, unwrap, swap, and fee-claim operations. The stable swap executes trades between USDC and USDC+ with fixed basis-point fees, and the proxy handles minting, burning, fee claims, and authority updates to ensure branded vault balances stay aligned with the oracle price.

Audit Scope

The assessment scope contains mainly the smart contracts of the proxy program for the *Reflect Proxy* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Reflect Proxy
  - Codebase: `https://github.com/palindrome-eng/reflect-proxy-program`
  - Commit Hash: 48d78c2c0f7a8703e3899236e470fc1e3f6b703d

We listed the files we have audited below:

- Reflect Proxy
  - src/state.rs
  - src/helpers.rs
  - src/proxy/create_proxy.rs
  - src/proxy/claim_fees.rs
  - src/proxy/wrap.rs
  - src/proxy/unwrap.rs
  - src/lib.rs
  - src/stable_swap/swap.rs
  - src/stable_swap/withdraw.rs

Findings

The security audit revealed:

- 1 critical issue
- 1 high issue
- 3 medium issues

- 5 low issues
- 1 informational issue

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	Incorrect branded_mint Amount Lead to User loss	Critical	Fixed
02	Crank Operation Failure Due to Over Claim Commission Fee	High	Fixed
03	Missing Slippage Checks in Several IXs	Medium	Fixed
04	DoS Risk When the Oracle Price Falls	Medium	Fixed
05	Possible Inflation Attack on Wrap IX	Medium	Fixed
06	Inconsistent Interest Split Calculation between Code Comment and Implementation	Low	Fixed
07	DoS Risk in CreateProxy Account Creation	Low	Fixed
08	Missing Overflow Check in Arithmetical Operation	Low	Fixed
09	Missing Validation on mint_to amount May Lead to User Loss	Low	Fixed
10	Potential Overflow in Intermediate Multiplication Despite Valid Final Result	Low	Fixed
11	Inconsistent Max Fee Check between Code and Comment	Informational	Fixed

4 Key Findings and Recommendations

4.1 Incorrect branded_mint Amount Lead to User loss

Severity: Critical	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

In `ProxyState::wrap`, the code that computes the amount of `branded_mint` to mint is as follows:

```
384     let amount = if branded_mint_supply == 0 {
385         // First deposit: mint 1:1 branded tokens
386         wrap_amount
387     } else {
388         // Subsequent deposits: maintain ratio based on existing supply
389         wrap_amount
390             .checked_mul(branded_mint_supply)
391             .and_then(|x| x.checked_div(self.principal()))
392             .ok_or(ProgramError::InvalidInstructionData)?
393     };
```

src/state.rs#L384-L393

In the `else` branch, three different units are mixed:

- `wrap_amount`: the actual amount of USDC+ tokens deposited by the user (6 decimals)
- `self.principal()`: the USDC value converted via the oracle
- `branded_mint.supply()`: the number of `branded_mint` tokens

The current formula directly uses `wrap_amount * supply / principal` to compute the user's `branded_mint` amount. Therefore, whenever the oracle price is not 1, the computed `branded_mint` amount will not match the true value.

Impact

- When the USDC+ `price > 1`, the user receives fewer `branded_mint` than expected.
- When the USDC+ `price < 1`, it over-mints, and the user receives more `branded_mint` than expected.

In either case, the conversion ratio is amplified or reduced by price fluctuations, which can cause direct losses to all participants.

Recommendation

It is recommended to first unify the units of `wrap_amount` and `ProxyState.principal` before computing the number of `branded_mint` to mint.

Mitigation Review Log

Fixed in the commit 34ae41c73b38b0ee3b5dc9fe5c0e1d794d153205.

4.2 Crank Operation Failure Due to Over Claim Commission Fee

Severity: High	Status: Fixed
Target: Smart Contract	Category: Pre

Description

In `ProxyState.claim_fees`, the code that computes the USDC+ amount corresponding to `integrator_commissions` works as follows:

```
527     let usdc_plus_value_of_principal = self
528         .principal()
529         .checked_mul(1_000_000)
530         .and_then(|x| x.checked_div(oracle_price))
531         .unwrap();
532
533     let usdc_plus_value_of_principal =
        ↪ TokenAccount::from_account_info(stablecoin_proxy_state_vault)?
534         .amount()
535         .checked_sub(usdc_plus_value_of_principal)
536         .unwrap();
```

src/state.rs#L527-L536

When converting `principal` from USDC value to USDC+ quantity, it uses integer division `self.principal() * 1_000_000 / oracle_price`, and the result is rounded down. It then subtracts this `usdc_plus_value_of_principal` from the vault's current total USDC+ to obtain the commissions amount payable to the integrator.

Because integer division floors the result, the converted `principal` is always slightly less than the actual USDC+ amount. As a result, the remainder from the subtraction is larger by a portion that in fact comes from `principal`, but this function transfers it to the integrator's token account as part of the commissions.

Impact

After the integrator calls `claim_fees`, the vault's actual USDC+ balance is less than the principal recorded in `ProxyState.principal()`. Assuming the oracle price stays the same or increases, at the next `crank` the calculation of `interest_generated` will underflow and panic (refer to the issue "DoS Risk When the Oracle Price Falls").

Recommendation

It is recommended to adjust the calculation here to ensure that, when computing the integrator's commissions, the vault still has enough USDC+ to cover the principal.

In addition, It is also recommended to rename the second `usdc_plus_value_of_principal` variable to reflect its true meaning, e.g., `usdc_plus_value_of_commissions`.

Mitigation Review Log

Fixed in the commit 34ae41c73b38b0ee3b5dc9fe5c0e1d794d153205.

4.3 Missing Slippage Checks in Several IXs

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: MEV Risk

Description

In the `unwrap` IX, the `usdc+` token amount to transfer to the user is calculated based on the oracle price. However, there is no slippage protection for the user's unwrap operation. If the oracle price fluctuates sharply after the user submits the TX, the `usdc+` token amount the user receives may differ from expectations.

A similar issue exists in the `wrap` and `swap` IXs.

Impact

If arbitragers identify an arbitrage opportunity, they can target this IX and, through sandwich or front running attacks, push the price up, causing the user to receive fewer `usdc+` tokens than expected.

Recommendation

It is recommended to add slippage checks in the `wrap` / `unwrap` / `swap` IXs to avoid unexpected, unnecessary losses for users.

Mitigation Review Log

Fixed in the commit e312d266e7d88b43f16391a63d8b3ac5b8586090 and f759ea9fbcc6d1d3f669da87643a8818a334a41e.

4.4 DoS Risk When the Oracle Price Falls

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Math

Description

In `ProxyState.crank`, `usdc_value` is computed from the oracle price and then used to calculate the new interest.

```
325     let interested_generated = usdc_value
326         .checked_sub(self.principal() + self.integrators_commission())
327         .unwrap();
```

src/state.rs#L325-L327

The oracle price changes dynamically and may go up or down. If the oracle price decreases between two cranks, the `checked_sub` subtraction above may fail and return an error.

Impact

A failed `crank` operation will directly prevent users from wrapping/unwrapping, and integrators' commission fees cannot be claimed. Users and integrators would then need to wait until the oracle price rises again to a level where the subtraction no longer fails before they can continue interacting with the program.

Recommendation

According to the documentation, the intended formula is:

`I_raw = max(0, V - (principal + commission))`

It is recommended to modify the interest calculation in `crank` to align with the formula above.

Mitigation Review Log

Fixed in the commit 34ae41c73b38b0ee3b5dc9fe5c0e1d794d153205.

4.5 Possible Inflation Attack on Wrap IX

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

In `ProxyState::wrap`, when `branded_mint_supply == 0`, the deposited USDC+ amount is directly used as the initial `branded_mint` amount to mint.

```
384     let amount = if branded_mint_supply == 0 {
385         // First deposit: mint 1:1 branded tokens
386         wrap_amount
387     } else {
```

src/state.rs#L384-L387

This minting method introduces a potential inflation attack:

Step 1. The attacker, as the first user to mint `branded_mint`, obtains all current shares of `branded_mint`.

Step 2. Since the vault is an SPL token account, after the initial mint and before the victim invokes `wrap`, the attacker can transfer a large amount of USDC+ to the vault via a normal transfer.

Step 3. When the victim calls `wrap`, `crank` first updates `principal` to reflect most of the vault's assets, while the number of new `branded_mint` to mint for the victim is still calculated as `wrap_amount * supply / principal`. Because the denominator is artificially inflated, the victim receives almost zero shares while paying the full amount.

Step 4. The attacker finally burns the shares minted initially and can redeem almost all assets in the vault.

Impact

Once exploited, the victim's entire deposit will be transferred to the attacker, resulting in a loss equal to the victim's actual deposit amount.

Recommendation

To mitigate inflation attacks, we recommend implementing one of the following strategies, in order of preference:

1. Dead Shares (Recommended): Permanently lock a portion of the initial shares to establish a baseline share supply that cannot be manipulated. Mint initial shares to a burn address or locked account, ensuring dead shares represent sufficient value to prevent manipulation. This follows proven patterns and is simple and effective for preventing zero-supply manipulation.

2. Initial Asset Deposits: This approach is similar to the dead shares, but instead of permanently locking a portion of the shares, it requires checking the minimum share limit on every increase or decrease in shares. Maintain a minimum threshold of underlying assets relative to total shares at all times. Although this method adds code complexity and computational overhead, it is suitable for scenarios where each position is susceptible to inflation attacks.
3. Re-calculate Effective Underlying Assets: When the share price becomes very high due to manipulation, users may experience significant precision loss when minting shares. For example, if 10,000 tokens are required per share, a user depositing 19,999 tokens would only receive 1 share, losing 9,999 tokens to rounding. To mitigate this, the protocol should calculate and return the exact number of underlying tokens needed to mint whole shares, then never take any excess tokens from the user. This ensures users only pay the precise amount needed (10,000 tokens for 1 share in the example) and eliminates value loss from precision rounding, though it requires additional logic to handle partial deposits.
4. Virtual Shares and Decimals Offset: Implement virtual accounting that increases precision and reduces rounding impacts through mathematical offsets. Add virtual shares and virtual assets to calculations, implement decimal precision offsets, and use higher precision arithmetic internally. This approach maintains precision without permanent value lock and offers flexible, tunable parameters, but is the most complex to implement correctly. For technical details, please refer to https://docs.openzeppelin.com/contracts/5.x/erc4626#defending_with_a_virtual_offset

The choice of mitigation strategy should be based on the protocol's specific requirements, risk tolerance, and tokenomics model.

Mitigation Review Log

Fixed in the commit 34ae41c73b38b0ee3b5dc9fe5c0e1d794d153205.

4.6 Inconsistent Interest Split Calculation between Code Comment and Implementation

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Precision

Description

In `ProxyState::crank` , the interest is split into `principal` and `integrators_commission` .

```
330     let generated_principal = interested_generated
331         .checked_mul(10_000 - self.fee() as u64)
332         .and_then(|x| x.checked_div(10_000))
333         .unwrap();
334
335     self.add_principal(generated_principal);
336
337     // Add the Integrators Commissionå
338     let generated_integrators_commission = interested_generated
339         .checked_sub(generated_principal)
340         .unwrap();
```

src/state.rs#L330-L340

However, the current calculation method attributes dust tokens caused by division precision to `integrators_commission`.

The implementation here also differs slightly from the comments.

```
316     /// Crank the proxy_state.principal value to be up to date.
317     ///
318     /// > Use the current USDC+/USDC conversion to calculate the current
        ↪ value of the USDC+ in the Vault.
319     /// > Calculate the Interested Generated since the last crank: Value
        ↪ of USDC+ in the Vault - (proxy_state.principal +
        ↪ proxy_state.integrators_commission)
320     /// > Calculate the Integrators Commission and add it to the
        ↪ proxy_state.integrators_commission: Interested Generated *
        ↪ proxy_state.fee / 10_000
321     /// > Calculate the Additional Principal and add it to the
        ↪ proxy_state.principal: Interested Generated * (1 -
        ↪ proxy_state.fee) / 10_000
```

src/state.rs#L316-L321

Impact

When `interested_generated` is very small, it may occur that `generated_principal == 0` while `generated_integrators_commission > 0`, so all new interest goes to the integrator.

Proof of Concept

Consider `fee = 100` and `interested_generated = 1`; the calculation yields:

- `generated_principal` => 0
- `generated_integrators_commission` => 1

All of `interested_generated` is allocated to `integrators_commission`.

Recommendation

It is recommended to readjust this part of the calculation to keep the code comments consistent with the implementation.

Mitigation Review Log

Fixed in the commit 34ae41c73b38b0ee3b5dc9fe5c0e1d794d153205.

4.7 DoS Risk in CreateProxy Account Creation

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

The `CreateProxy` IX creates a PDA account called `proxy_state`. This account is created using `pinocchio_system::instructions::CreateAccount`.

However, the IX does not account for the possibility that the PDA might already have some lamports. Since `CreateAccount` will fail if the destination account already holds any lamports, directly calling it can cause the IX to revert.

Impact

An attacker could exploit this by transferring a small amount of lamports to the `proxy_state` account before the integrator invokes `CreateProxy` IX, causing the `CreateAccount` IX to fail and preventing the account from ever being properly created.

Recommendation

It is recommended to use the fixed `CreateProxy.process` utility function to create `proxy_state`.

Here is a reference link from the SPL Implementation: `program/src/tools/account.rs#L19-L73`

Mitigation Review Log

Fixed in the commit 34ae41c73b38b0ee3b5dc9fe5c0e1d794d153205 and f759ea9fbcc6d1d3f669da87643a8818a334a41e.

4.8 Missing Overflow Check in Arithmetical Operation

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Code QA

Description

The program does not enable the `overflow-checks` feature in the global `Cargo.toml`. Therefore, in release mode, all integer operations performed directly with operators have no corresponding overflow checks.

Here is one site in the program where overflow may occur:

```
325     let interested_generated = usdc_value
326         .checked_sub(self.principal() + self.integrators_commission())
327         .unwrap();
```

src/state.rs#L325-L327

The `principal` and `integrators_commission` are both `u64`, but even if their sum exceeds `u64::MAX`, it will not raise an overflow error here.

Impact

Unexpected arithmetic overflow may lead to unexpected program behavior.

Recommendation

It is recommended to ensure:

- Enable the `overflow-checks` feature in the global `Cargo.toml`, at least ensuring that the release build of the program includes runtime overflow checks.
- Ensure that all calculations using operators cannot overflow; otherwise, safer `checked_*` methods should be used to detect and report potential runtime errors.

Mitigation Review Log

Fixed in the commit f759ea9fbcc6d1d3f669da87643a8818a334a41e.

4.9 Missing Validation on mint_to amount May Lead to User Loss

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

If the user's wrap amount is very small (e.g., 1), such that `amount * branded_mint_supply < principal`, this will cause the computed `mint_to` amount in `ProxyState.wrap` to be 0.

Impact

In this case, the stable_coin the user transfers to the vault via `wrap` will not mint the corresponding `branded_mint`, and the user directly loses that portion of stable_coin.

Recommendation

It is recommended to add a check to ensure the `mint_to` amount is greater than 0; otherwise, reject such wrap operations.

Mitigation Review Log

Fixed in the commit 90a2e7fbd077df3f566c4855aac237bc1332dab8.

4.10 Potential Overflow in Intermediate Multiplication Despite Valid Final Result

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Code QA

Description

The following code snippets demonstrate common patterns in the program where expressions of the form `a * b / c` are evaluated using `checked_mul` for multiplication and `checked_div` for division to avoid overflow errors:

```
372  let vault_usdc_value =
     ↪ TokenAccount::from_account_info(stablecoin_proxy_state_vault)?
373      .amount()
374      .checked_mul(oracle_price)
375      .and_then(|x| x.checked_div(1_000_000))
376      .unwrap();
```

src/state.rs#L372-L376

```
389    wrap_amount
390        .checked_mul(branded_mint_supply)
391        .and_then(|x| x.checked_div(self.principal()))
392        .ok_or(ProgramError::InvalidInstructionData)?
```

src/state.rs#L389-L392

However, if the intermediate multiplication result `a * b` exceeds the maximum value representable by a `u64`, `checked_mul` will return `None` and cause the transaction to revert, even when the final result after division (`(a * b) / c`) would fit within the valid `u64` range.

Impact

This issue can cause valid transactions to fail unnecessarily due to integer overflow in intermediate calculations. It undermines the program's correctness and may degrade user experience by rejecting legitimate computations that only overflow temporarily during intermediate steps.

Recommendation

It is recommended that intermediate calculations use a wider integer type such as `u128` to accommodate the potentially larger intermediate values. Explicitly casting operands to `u128` before multiplication can effectively prevent overflow in the intermediate step.

Mitigation Review Log

Fixed in the commit 34ae41c73b38b0ee3b5dc9fe5c0e1d794d153205 and f759ea9fbcc6d1d3f669da87643a8818a334a41e.

4.11 Informational and Undetermined Issues

Inconsistent Max Fee Check between Code and Comment

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

In the code comments, it states regarding the fee check: "no need to check since if it's greater than 10,000 it will fail in the `set_fee()` function." However, in `ProxyState.set_fee`, the requirement is `fee < 10,000`.

```
224     pub fn set_fee(&mut self, fee: u16) -> Result<(), ProgramError> {
225         if fee.ge(&MAX_FEE_BPS) {
226             return Err(ProgramError::InvalidAccountData);
227         }
```

src/state.rs#L224-L227

It is recommended to keep the fee requirement consistent between the code and the comments.

Fixed in the commit 90a2e7fbd077df3f566c4855aac237bc1332dab8.

5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

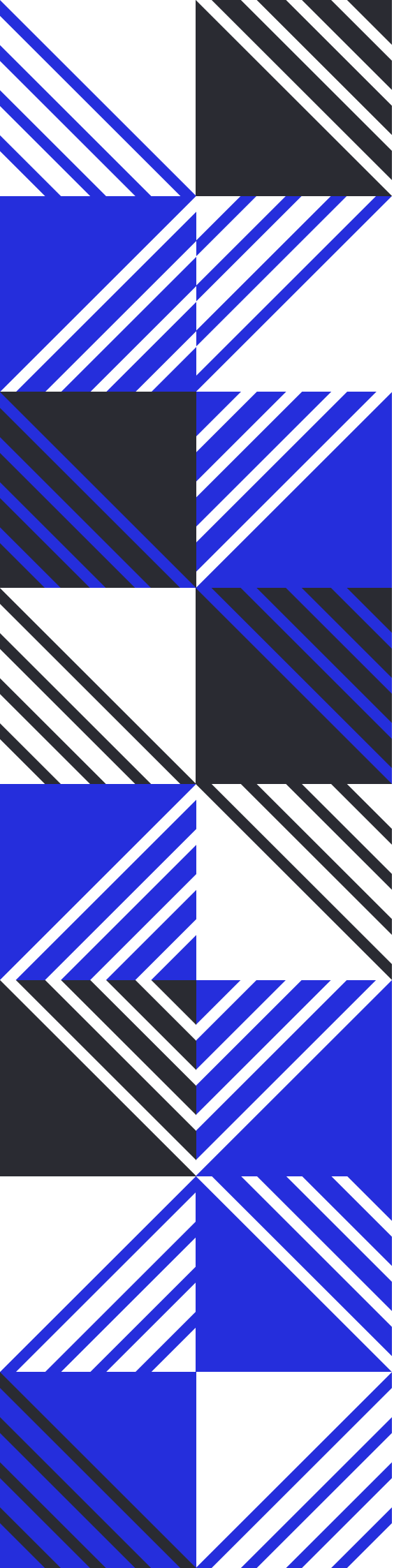
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs